

Data Structure and Algorithm, Spring 2026

Homework 1

Due: 13:00:00, Tuesday, March 24, 2026

TA E-mail: dsa_ta@csie.ntu.edu.tw

Rules and Instructions

- Any form of cheating, lying, or plagiarism will not be tolerated. Students can get zero scores and/or fail the class and/or be kicked out of school and/or receive other punishments for those kinds of misconducts.
- Discussions on course materials and homework solutions are encouraged. But you should write the final solutions alone and understand them fully. Books, notes, and Internet resources (including chatGPT) can be consulted, but not copied from.
- Since everyone needs to write the final solutions alone, there is **absolutely no need to lend your homework solutions and/or source codes to your classmates at any time**. In order to maximize the level of fairness in this class, lending and borrowing homework solutions are both regarded as dishonest behaviors and will be punished according to the honesty policy.
- We will detect and penalize plagiarism with the criteria of **line-by-line similarity** in any part of your code/writing, when compared with the code/writing from any student or any external resource. Exceptions will be given to those showing **time-stamped records** that the solution comes from a meaningful interactive discussion with human and/or AI tools, **rather than copying from** others.
- In Homework 1, the problem set contains 4 problems. The set is divided into two parts, the non-programming part (Problems 1, 2) and the programming part (Problems 3, 4).

- For your solutions in the non-programming part, unless otherwise noted, there are two common requirements:
 - Your answer should be as simple as possible. For instance, $\Theta(\frac{1}{n} + \lg(n^{541}) + 8763n^2)$ should be written as $\Theta(n^2)$.
 - Your answer should also come with a *brief* explanation. The word *brief* means “very short.” An example of a *brief* explanation is as follows.

Sample problem: Write down the time complexity for this algorithm using the Θ notation along with an expression of n .

HELLO-WORLD(n)

```
1  for  $i = 1 \dots 2n$ 
2      for  $j = 1 \dots 2n$ 
3          print("Hello world")
```

Sample solution: The iteration counts of the two loops are $2n$ and $2n$ respectively, so there are $4n^2$ iterations. Hence, the time complexity is $\Theta(n^2)$.

□

- For problems in the non-programming part, you should combine your solutions in *one* PDF file. Your file should generally be legible. Your solution must be as simple as possible. At the TAs' discretion, solutions which are too complicated can be penalized or even regarded as incorrect. If you would like to use any theorem which is not mentioned in the classes, please include its proof in your solution.
- The PDF file for the non-programming part should be submitted to Gradescope as instructed, and you should use Gradescope to tag the pages that correspond to each subproblem to facilitate the TAs' grading. **Failure to tag the correct pages of the subproblem can cost you a 10% penalty.**

- For problems in the programming part, you should write your code in C programming language, and then submit the code via the *DSA Judge+*: <https://dsa.csie.ntu.edu.tw/>. Each day, you can submit up to 10 times for each problem. The judge system will compile your code with

```
gcc [id].c -O2 -std=c17
```

- For debugging in the programming part, we strongly suggest you produce your own test-cases with programs and/or use tools like `gdb` or compile with flag `-fsanitize=address` to help you solve issues like illegal memory usage. For `gdb`, you are strongly encouraged to use compile flag `-g` to preserve symbols after compiling.

Note: For students who use IDE (VScode, DevC++...) you can modify the compile command in the settings/preference section. Below are some examples:

```
- gcc [id].c -O2 -std=c17 -fsanitize=address
  # Works on Unix-like OS, but possibly not M1/M2
- gcc [id].c -O2 -std=c17 -g # use it with gdb
```

- The score of the part that is submitted after the deadline will get some penalties according to the following rule:

$$Late\ Score = \max \left(\left(\frac{5 \times 86400 - Delay\ Time(sec.)}{5 \times 86400} \right) \times Original\ Score, 0 \right)$$

As mentioned at the beginning of the class, you have **eight gold medals** this semester, each of which can be used to extend the deadline of **one** programming problem by **24 hours**. You can choose how to allocate the medals around the end of the semester.

- If you have questions about this homework, please go to the Discord channel and discuss the issues with your classmates. This is *strongly preferred* because it will provide everyone with a more interactive learning experience. If you really need email clarifications, you are encouraged to include the following tag in the subject of your email:
 - Please include the following tag in the subject of your email: "[HW1.Px]", specifying the problem where you have questions. For example, "[HW1.P2] Is k in subproblem 5 an integer?". Adding the tag allows the TAs to track the status of each email and to provide faster responses to you.
 - If you want to provide your code segments to us as part of your question, please upload it to [Gist](#) or similar platforms and provide the link. Please remember to protect your uploaded materials with proper permission settings. Screenshots or code segments directly included in the email are discouraged and may not be reviewed.

Problem 1 - Data Search and Analysis (100 pts)

1. (15 pts, 5 pts each) For each pair of functions $(f(n), g(n))$, determine which of the following statement is true:

- $O(f(n)) \subsetneq O(g(n))$
- $O(g(n)) \subsetneq O(f(n))$
- $O(f(n)) = O(g(n))$
- none of the above

For each pair, please justify your solution with ≤ 5 lines of brief explanations.

- (a) • $f(n) = \log_2 n$
• $g(n) = \log_3 n$
- (b) • $f(n) = 2^n$
• $g(n) = 3^n$
- (c) • $f(n) = n^n$
• $g(n) = n!$

2. (10 pts) Now, define $f(m, n) = O(g(m, n))$ iff there exists positive constants (c, m_0, n_0) such that for all $m \geq m_0$ and $n \geq n_0$, $f(m, n) \leq c \cdot g(m, n)$. For the two functions

- $f(m, n) = m + n$
- $g(m, n) = m \log_2 n$

please make a formal **proof** from the definition, with unlimited number of lines (while we still hope that your solution is concise), about which of the following statement is true.

- $O(f(m, n)) \subsetneq O(g(m, n))$
- $O(g(m, n)) \subsetneq O(f(m, n))$
- $O(f(m, n)) = O(g(m, n))$
- none of the above

3. For each sub-sub-problem, please analyze the time complexity, and justify your solution within ≤ 20 lines of brief explanations.

- (a) (5 pts) Given the DISCREET-SORT algorithm below, analyze its best-case time complexity.

DISCREET-SORT(arr)

```
1   $i \leftarrow 0$ 
2   $n \leftarrow \text{length}(arr)$ 
3  while  $i < n$ 
4      if  $i = 0$  or  $arr[i - 1] \leq arr[i]$ 
5           $i \leftarrow i + 1$ 
6      else
7          SWAP( $arr[i], arr[i - 1]$ )
8           $i \leftarrow i - 1$ 
```

- (b) (10 pts) Given the DISCREET-SORT algorithm above, analyze its worst-case time complexity.
- (c) (15 pts) Given the recursive algorithm below, analyze its worst-case time complexity.

DOUBLE-TWODOWN(n)

```
1  PRINT( $n$ )
2  if  $n > 0$ 
3      DOUBLE-TWODOWN( $n - 2$ )
4      DOUBLE-TWODOWN( $n - 2$ )
5      DOUBLE-TWODOWN(0)
```

4. (15 pts) Prove or disprove the claim below. Either way, you should provide a *formal proof*. That is, if you believe the claim to be true, prove the claim by specifying all constants like c and n_0 or using any theorems that have been proved in class; if you believe the claim to be false, disprove the claim by providing a correct counterexample and/or illustrate with other logical arguments. **Please note that the Convergence “Theorem” made in class was meant for Θ instead of O . So it may or may not be applied here unless you make a clear logical argument.**

Claim: For any $p \in \mathbb{N}$ and any $q > 1$, $n^p = O(q^n)$.

5. (30 pts) In this sub-problem, we will provide you with a story version, and an equivalent “unflavored” version. Please feel free to decide what version(s) you want to read to solve the sub-problem.

Story version for Dormitory Snack Allocation (DSA):

Sam bought some snacks and brought them back to zir dorm, splitting them into n piles. The i -th pile contains `piles[i]` pieces. There are exactly h days before the snacks expire. Sam chooses an eating speed of k pieces per day. During each day, Sam eats from exactly one pile, consuming up to k pieces from that pile. If the pile contains fewer than k pieces, Sam finishes the pile and does not continue to another pile on the same day.

Please design an algorithm that returns the minimum integer k such that all snack piles can be finished within h days, with (worst-case) **time complexity** $O(n \log K)$, **where** $K = \max_{1 \leq i \leq n}(\text{piles}[i])$. Express your algorithm with pseudocode, and use ≤ 30 lines to briefly explain why your algorithm is correct and why it meets the required time complexity.

Unflavored version:

Given a positive integer array `piles` of length $n > 0$, and a positive integer h . Design an algorithm that finds the minimum positive integer k such that

$$\sum_{i=1}^n \left\lceil \frac{\text{piles}[i]}{k} \right\rceil \leq h,$$

with (worst-case) **time complexity** $O(n \log K)$, **where** $K = \max_{1 \leq i \leq n}(\text{piles}[i])$. Express your algorithm with pseudocode, and use ≤ 30 lines to briefly explain why your algorithm is correct and why it meets the required time complexity.

Problem 2 - Dual Structures Adventure (100 pts)

1. (15 pts) We've introduced Insertion Sort in class, where it is implemented using an array. Now, given the pseudocode for Insertion Sort on an array below, please write the pseudocode for Insertion Sort on a singly linked list. You can assume that the singly linked list is valid.

INSERTION-SORT(A)

```
1  for  $i = 1$  to  $A.Length$  :  
2      INSERT( $A, i$ )  
3  return  $A$ 
```

INSERT(A, m)

```
1   $data = A[m]$   
2   $i = m - 1$   
3  while  $i > 0$  and  $A[i] > data$  :  
4       $A[i + 1] = A[i]$   
5       $i = i - 1$   
6   $A[i + 1] = data$ 
```

Your algorithm should take $O(n^2)$ time and $O(1)$ space to sort the linked list, just as Insertion Sort does on an array. You do **not** need to explain why the pseudocode is correct.

Hint: Since it is relatively more complex to traverse from the back to the front in a singly linked list, you may consider traversing the list in another direction when inserting an element. :)

2. (15 pts) Given an array of length $(2m - 1)$, containing every integer from 1 to m **exactly twice** except for one integer, which appears only once, write the pseudocode for an algorithm that finds the integer appearing only once in $O(m)$ time and $O(1)$ space. Briefly explain why your solution is correct and why it meets the time and space constraints with ≤ 5 lines.

3. (20 pts) Given the head of a singly linked list with an even length of $(2m)$

$$L_1 \rightarrow L_2 \rightarrow \dots \rightarrow L_{2m-1} \rightarrow L_{2m}$$

We want reorder the list to

$$L_1 \rightarrow L_{2m} \rightarrow L_2 \rightarrow L_{2m-1} \rightarrow L_3 \rightarrow L_{2m-2} \rightarrow \dots \rightarrow L_m \rightarrow L_{m+1}$$

Write down the pseudocode for an algorithm that can reorder the list accordingly in $O(m)$ time and $O(1)$ space. You may **not** modify the value of the nodes directly, and note that you do **not** know the value of m or $(2m)$ beforehand. Briefly explain why your solution is correct and why it meets the time and space constraints with ≤ 20 lines.

4. (20 pts) You are given an array *arr* of length n that contains only multiples of 6 and/or multiples of 7 (there are no elements in the array that are neither).

Let m be the total number of multiples of 6, and k be the total number of multiples of 7 in the array (where $0 \leq m, k \leq n$). Your task is to rearrange the array in-place, under $O(n)$ time and $O(1)$ space constraints, such that the first m elements are all multiples of 6, and the last k elements are all multiples of 7.

A student has written the following pseudocode (assuming 1-based indexing) to solve this problem:

REORDER-SIXSEVEN(*arr*, n)

```

1  left_bound = 1
2  for i = 1 to n
3      if arr[i] mod 6 = 0 :
4          SWAP(arr[i], arr[left_bound])
5          left_bound = left_bound + 1
6  k = 1
7  for i = 1 to left_bound - 1 :
8      if arr[i] mod 7 = 0
9          SWAP(arr[i], arr[k])
10         k = k + 1
11  return arr
```

Now, please examine the pseudocode above and determine whether the algorithm is correct.

- If you think the pseudocode is correct, provide a correctness proof.
- If you think the pseudocode is incorrect, provide a counterexample.

5. (30 pts) Given an $m \times n$ matrix represented as a singly linked list, from top to bottom and from left to right. For example, a 3×4 matrix:

$$\begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \end{bmatrix}$$

is represented as:

$$a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow \dots \rightarrow l$$

Now, please **rotate** the matrix by 90 degrees counter-clockwise, so the matrix in the example becomes

$$\begin{bmatrix} d & h & l \\ c & g & k \\ b & f & j \\ a & e & i \end{bmatrix}$$

which should then be represented as

$$d \rightarrow h \rightarrow l \rightarrow c \rightarrow g \rightarrow k \rightarrow b \rightarrow f \rightarrow j \rightarrow a \rightarrow e \rightarrow i$$

The reordering must be done with $O(mn)$ time and $O(n)$ space, **without** generating new nodes. That is, the reordering must be performed in-place using the original nodes. Write down the pseudocode to achieve the reordering. Briefly explain why your solution is correct and why it meets the time and space constraints with ≤ 20 lines.

Problem 3 - Deploying Stochastic Advantage (100 pts)

Problem Description

When using linked lists to store ordered elements, a big disadvantage (over ordered arrays) is that searching for an element cannot be done efficiently with binary search because it is difficult to efficiently access the “mid-point” of a linked list. The skip list is here for rescue! In this problem, we ask you to play with the skip list. The following description of the skip list is modified from Wikipedia (http://en.wikipedia.org/wiki/Skip_list):

A skip list is built in layers. The bottom (zero-th) layer is an ordinary ordered linked list. Each higher layer acts as an “express lane” for the lists below, where a node in layer i appears in its upper layer $i + 1$ with probability $\frac{1}{2}$ (We flip a coin each time to decide whether it will appear in the upper layer or not).

A search for a target node begins at the head in the top list, and proceeds horizontally until the current node is greater than or equal to the target. If the current node is equal to the target, it has been found. If the current node is greater than the target, or the search reaches the end of the linked list, the procedure is repeated after returning to the previous node and dropping down vertically to the next lower list.

In summary, a skip list is an enhanced version of an **ordered linked list**. By introducing randomization, it improves the search time complexity from $O(n)$ to an *expected* time of $O(\log n)$. Since each node appears in the upper layer with a probability of $\frac{1}{2}$, layer $i + 1$ is expected to contain half as many nodes as layer i . Therefore, searching from the top layer downwards mimics the halving process of a binary search, effectively reducing the search space by half (albeit a different half than the shortcut we take for the ordered array) at each step.

We emphasize that the *expected* search time is $O(\log n)$, even though a worst-case time of $O(n)$ is theoretically possible. Typically, we use $O(n)$ to describe the worst-case complexity because there exists a specific input that will **always make it slow**. However, for a skip list, the efficiency is governed by random coin flips. Therefore, for any given input, the search time can be considered $O(\log n)$ in practice.

TODO

You need to implement the *reversed* ordered skip list. That is, the skip list is ordered from the largest number to the smallest one. Here are the 4 functions that we ask you to implement:

FAST-GET($L, data$)

```
1  node = L.head[L.fastLayers] // the top layer's dummy head
2  while node ≠ NIL
3      while node.next ≠ NIL and data ≤ node.next.value
4          node = node.next
5      if node.below == NIL and node.value == data
6          // Always returns the target in the bottom layer
7          return node
8      node = node.below
9  return NIL
```

SLOW-GET($L, data$)

```
1  node = L.head[0] // the bottom layer, which is simply a reversely ordered linked list
2  while node ≠ NIL and data ≤ node.value
3      if node.value == data
4          return node
5      node = node.next
6  return NIL
```

INSERT($L, data$)

Insertion can be done by first taking FAST-GET while storing the “previous” node of each layer. Then, one can insert the new node after the “previous” node of each layer from bottom to top according to the result of the coin flip. Note that if a node is inserted into the top layer, then an additional coin flipping is needed to see if one more layer should be added.

REMOVE($L, data$)

Removal can be done by first taking FAST-GET and removing the target nodes (in all layers) once they have been found. Note that if all nodes in the top layer are removed, the top layer should also be removed by updating $L.fastLayers$.

We strongly encourage using a dummy head for each linked list to facilitate your implementation, but you need to design the value of the dummy head so that the skip list can work well.

To make insertion (and hence the whole skip list) deterministically judge-able, the randomized coin flip results will be given to you in the input. You must use the results to implement your skip list. For more details, please refer to the input section.

Input

The first line has one integer M representing the number of operations. The second line has a binary string C represents the result of the coin flip. The i -th bit C_i is the result of the i -th coin flip. The next M lines each has two integers t, k separated by a space. t represents the type of operation, and k represents the additional parameter for that operation.

- $t = 1$ is for the FAST-GET operation. You need to find the smallest number that is greater than or equal to k by running the FAST-GET function described above.
- $t = 2$ is for the SLOW-GET operation. You need to find the smallest number that is greater than or equal to k by running the SLOW-GET function described above.
- $t = 3$ is for the INSERT operation. You are required to insert k into the skip list.
- $t = 4$ is for the REMOVE operation. You are required to remove k from the skip list. If k is not in the skip list, simply ignore it and proceed to the next operation.

When performing INSERT, if the coin flip result is 1, then the node should be promoted to the upper layer. You need to repeat the promotion process until the result of the coin flip is 0. That is, the INSERT process will stop only when the coin flip result is 0. Note that to promote or not depends solely on the coin flip, therefore a single insertion can cause the skip list to grow more than one layers.

Output

For each operation of type $t = 1$ or $t = 2$, you should print some numbers separated by spaces on a single line. Specifically, print all *node.value* satisfying $k \leq \text{node.value}$ (excluding any dummy head you choose to implement) in the search order. If no values meet the condition, print -1 on a single line. Note that a value may be printed multiple times if it appears in multiple layers for type 1 operations.

Constraints

- $1 \leq M \leq 10^5$
- $t \in \{1, 2, 3, 4\}$
- $0 \leq k \leq 10^{18}$
- $C_i \in \{0, 1\}$ for all i and is in principle “sufficiently random” (i.e. won’t bring your program to the worst cases of the skip list)
- The number of bits in C is sufficient for correct implementation and is strictly less than 10^7 .

- It is ensured that there are no duplicate numbers in the skip list at any given time, but a number may be re-inserted after being removed.
- The number of layers in the skip list is always strictly less than 100.

Subtasks

Subtask 1 (20 pts)

- $t \in \{2, 3, 4\}$
- $1 \leq M \leq 3 \times 10^3$
- The two constraints mean that the task can be solved with the bottom layer of a skip list, a typical linked list, or even an array.

Subtask 2 (40 pts)

- $t \in \{1, 3\}$, i.e., skip list without REMOVE

Subtask 3 (40 pts)

- $t \in \{1, 3, 4\}$, i.e., a fully functional skip list

Sample Testcases

Sample Input 1

```
6
0100
3 7
3 14
3 49
2 10
2 7
2 50
```

Sample Output 1

```
49 14
49 14 7
-1
```

Sample Input 2

8
10110010
3 0
3 100
1 15
3 20
1 0
3 50
1 25
1 20

Sample Output 2

100 100 100
100 100 0 0
100 100 50 50
100 100 50 50 20

Sample Input 3

10
11100100
3 100
3 15
3 50
1 20
4 100
1 20
3 100
1 20
4 50
1 0

Sample Output 3

100 100 100 50 50
50 50
50 50
100 15

Problem 4 - Divine Stele Ascend (100 pts)

Problem Description

On a mountain, there are n steles numbered from 1 to n . The altitude of the i -th stele is h_i . It is guaranteed that there exists an integer $2 \leq k \leq n - 1$ such that

- $h_1 \leq h_2 \leq \dots \leq h_k$.
- $h_k \geq h_{k+1} \geq \dots \geq h_n$.

You will be given an integer T , the number of days. Each day, exactly one of the following operations is performed:

- Type 1. Given integers L, R, M . Among the steles with indices from L to R , choose m that solves

$$\operatorname{argmin}_{L \leq m \leq R} \sum_{i=L}^R (h_m - h_i)^2$$

Then, place M magic stones on stele m . If multiple steles achieve the minimum value, choose the smallest m within those steles.

- Type 2. Given integers H, M . Fly to altitude H and place M magic stones on every stele whose altitude is H .
- Type 3. Given an integer S . Move to stele S , take all magic stones on it, and output the number of magic stones taken.
- Type 4. Given an integer H . Fly to altitude H , take all magic stones from every stele whose altitude is H , and output the total number of magic stones taken.

Input

The first line contains 2 space-separated integers n and T . The second line contains n integers, representing the height of steles $h_1, h_2, \dots, h_n$. The following T lines contains one, two, three, or four space-separated integers depending on the operation:

- 1 L R M , indicating a type 1 operation with parameters L , R and M .
- 2 H M , indicating a type 2 operation with parameters H and M .
- 3 S , indicating a type 3 operation with parameter S .
- 4 H , indicating a type 4 operation with parameter H .

Output

- For each type 1 operation, do not print out anything.
- For each type 2 operation, do not print out anything.
- For each type 3 operation, print the number of magic stones taken in a line.
- For each type 4 operation, print the total number of magic stones taken in a line.

Constraints

- $3 \leq n \leq 10^6$
- $1 \leq T, M \leq 10^6$
- $1 \leq h_i \leq 10^9$
- $1 \leq H \leq 10^9$
- $1 \leq L, R, S \leq n$
- Each operation is guaranteed to be legitimate.

Hint

While the problem may look artificial (but hopefully fun), it aims to teach an important lesson: you can choose to transform and/or store your information (e.g., the number of magic stones at each stele) in different ways, possibly even in separate arrays or data structures, to speed up certain operations. For instance, storing some transformed information derived from h_i may make type-1 operations applicable to the bxxxxxy-sxxxxh algorithm.

Subtasks

Subtask 1 (10 pts)

- $3 \leq n \leq 1000$
- $1 \leq T \leq 1000$
- $1 \leq M \leq 1000$
- $1 \leq h_i \leq 1000$
- $1 \leq H \leq 1000$
- $1 \leq L, R, S \leq n$

Subtask 2 (30 pts)

- Only operations of type 1, type 2, or type 3.
- $3 \leq n \leq 10^6$

- $1 \leq T, M \leq 10^6$
- $1 \leq h_i \leq 10^9$
- $1 \leq H \leq 10^9$
- $1 \leq L, R, S \leq n$

Subtask 3 (60 pts)

No other constraints.

Sample Testcases

Sample Input 1

```

16 16
1 2 2 3 3 3 5 7 7 6 5 3 3 2 1 1
1 1 16 5
1 1 1 10
1 1 4 50
1 14 15 100
2 3 10
3 4
3 5
1 4 5 2
4 2
4 3
3 2
2 2 50
2 5 7
1 2 3 9
3 2
4 2

```

Sample Output 1

```

15
10
150
32
0
59
100

```

Sample Input 2

11 10
15 15 20 20 20 16 16 16 15 15 15
2 15 1
1 9 11 10
1 6 11 100
4 15
1 3 5 20
1 1 11 999
4 15
3 6
1 3 8 66
4 20

Sample Output 2

15
0
1099
86